

OpenShift Workshop

1 - Manage Containers with Podman

Mihai Criveti, Principal Architect at IBM. STSM, RHCA III, CKA/CKAD/CKS

February 7, 2023

@CrivetiMihai



Red Hat

OpenShift Workshop

Workshop Overview

Part 1

- Install podman, buildah and skopeo

Part 2

- Publish images to external registries
- Quay.io and Clair

Part 3

1. Install CodeReady Containers, Create a project called wordpress
2. Create users and groups and setup httpassword authentication
3. Deploy mysql from registry.redhat.io/rhel8/mysql-80 and configure the secret
4. Deploy wordpress from docker.io/library/wordpress:5.5.0-php7.2-apache
5. Create a route and test wordpress, scale out

- <https://podman.io/getting-started/>
- <https://github.com/containers/buildah>
- <https://github.com/containers/skopeo>
- <https://developers.redhat.com/products/rhel/ubi>
- <https://quay.io/>
- <https://www.katacoda.com/courses/containers-without-docker/running-containers-with-podman>
- <https://developers.redhat.com/products/codeready-containers/overview>

Podman, Skopeo and Buildah

Podman Overview

What is Podman?



Figure 1: podman - manage pods, containers and OCI compliant container images

How is Podman different?

- Can be run as a regular user without requiring root.
- Can manage **pods** (groups of one or more containers that operate together).
- Lets you import Kubernetes definitions using `podman play`.
- Fork-exec model instead of client-server model (containers are child processes of podman).
- Compatible with Docker, Docker Hub or any OCI compliant container implementation.

What is Buildah?



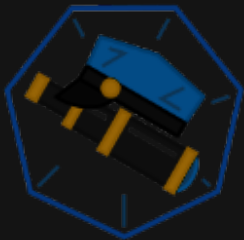
buildah

Figure 2: buildah - build container images from CLI or Dockerfiles

How is Buildah different?

- Containers can be build using simple CLI commands or shell scripts instead of Dockerfiles.
- Images can then be pushed to any container registry and can be used by any container engine, including Podman, CRI-O, and Docker.
- Buildah is also often used to securely build containers while running inside of a locked down container by a tool like Podman, OpenShift/Kubernetes or Docker.

What is Skopeo?



skopeco

Figure 3: skopeo - inspect and copy containers and images between different storage

How does Skopeo help?

- It can copy images to and from a host, as well as to other container environments and registries.
- Skopeo can inspect images from container image registries, get images and image layers, and use signatures to create and verify images.

Red Hat Image Sources Explained

Red Hat Software Collections Library (RHSC)

- For developers that need the latest versions of tools not in the RHEL release schedule.
- Use the latest development tools without impacting RHEL.
- Available to all RHEL subscribers.

Red Hat Container Catalog (RHCC)

- Certified, curated and tested images built on RHEL.
- Images have gone through a QA process.
- Upgraded on a regular bases to avoid security vulnerabilities.

Quay.io

- Public / private container repository.

Universal Base Image - UBI

Red Hat Universal Base Image - UBI

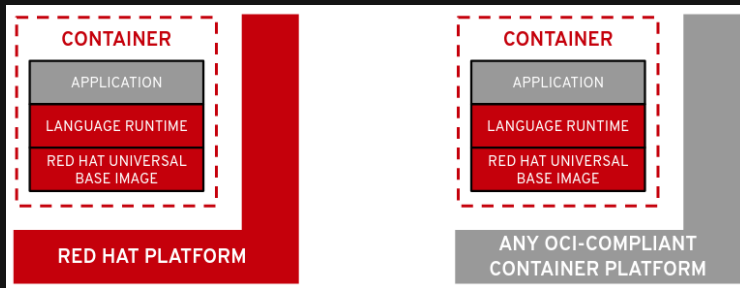


Figure 4: UBI - Freely distributable OCI compliant secure container base images based on RHEL

How does UBI Help?

- More than just a base image, UBI provides three base images across RHEL 7 and RHEL 8: `ubi`, `ubi-minimal` and `ubi-init`
- And a set of language runtimes (ex: `nodejs`, `ruby`, `python`, `php`, `perl`, etc.)
- All packages in UBI come from RHEL channels and are supported on RHEL and OpenShift.
- Secure by default, maintained and supported by Red Hat.

The Red Hat Container Catalog

Certified container images from Red Hat and 3rd party vendors

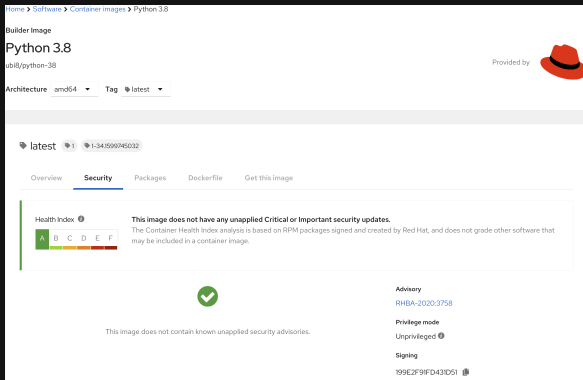


Figure 5: Container Images with a Container Health Index

Pulling a container image

```
podman pull registry.access.redhat.com/ubi8/python-38
```

What is podman-compose?

- An implementation of docker-compose with Podman backend.

Why podman-compose and when to use it?

- run unmodified `docker-compose.yaml` files, rootless
- no daemon or setup required
- Only depends on podman, Python 3 and PyYAML.

When NOT to use podman-compose?

- When you can use `podman pod` or `podman generate` and `podman play` instead to create pods or import Kubernetes definitions.
- For single-machine development, consider [CodeReady Containers](#)
- For multi-node clusters, check out Red Hat OpenShift, Kubernetes or [OKD](#).

Install podman, buildah and skopeo

Fedora 32 / RHEL 8

```
# Install podman, buildah and skopeo on Fedora 32  
sudo dnf -y install podman buildah skopeo slirp4netns fuse-overlayfs
```

Ubuntu / Debian

```
sudo apt update && sudo apt -y install podman buildah skopeo
```

Getting help

```
podman version  
podman --help      # list available commands  
man podman-ps      # or commands like run, rm, rmi, image, build  
podman info        # display podman system information  
https://podman.io/getting-started/installation
```

Rootless Containers and cgroup v2

Note that our regular user has UID 1000

```
uid=1000(cmihai) gid=1000(cmihai) groups=1000(cmihai)
```

What are UIDs mapped to inside the container?

```
podman unshare cat /proc/self/uid_map
```

```
0          1000          1
```

```
1          100000        65536
```

UID 0 is mapped my UID (1000). UID 1 is mapped to 100000, UID 2 would map to 100001, etc. That means that a container UID of 27 would map to UID 1000026.

Let's test this

```
mkdir test && podman unshare chown 27:27 test
```

```
ls -ld test
```

```
drwxrwxr-x. 2 100026 100026 4096 Sep 27 09:38 test
```

Running Containers with Podman

Searching for Images with `podman search`

Configure search sources

```
grep search /etc/containers/registries.conf
```

```
unqualified-search-registries =
```

```
['registry.fedoraproject.org', 'registry.access.redhat.com',  
'registry.centos.org', 'docker.io']
```

Searching for images (with filters)

```
podman search httpd --filter=is-official
```

INDEX	NAME	DESCRIPTION	STARS	OFFICIAL
docker.io	docker.io/library/httpd	The Apache HTTP Server	3181	[OK]

Adding a local registry configuration

Create a configuration file

```
mkdir -p ~/.config/containers
```

Add public and private registries in search order

```
vim $HOME/.config/containers/registries.conf
```

```
[registries.search]
```

```
registries = ["registry.access.redhat.com", "quay.io", "docker.io"]
```

```
[registries.insecure]
```

```
registries = ['localhost:5000']
```

Inspecting and pulling images

Inspecting Images with skopeo (ex: listing tags)

```
skopeo inspect docker://docker.io/library/httpd
```

Inspect the image with podman and show image history

```
podman inspect httpd:2.4.46-alpine  
podman history httpd:2.4.46-alpine
```

Pulling Images locally with podman pull

```
podman pull docker.io/library/httpd:2.4.46-alpine  
podman images
```

Running containers in interactive mode

Run an interactive session

```
podman run --name ubuntu --hostname ubuntu \  
--interactive --tty ubuntu /bin/bash
```

Reattach

```
podman start --attach --interactive ubuntu
```

Delete your container on exit

```
podman run --rm --name ubuntu --hostname ubuntu \  
--interactive --tty ubuntu /bin/bash
```

Running Containers in the background

Run a sample httpd container to serve a webpage

```
# Run a container in the background, bind port 8080
```

```
podman run --name httpd --detach \  
  --publish 8080:8080/tcp \  
  registry.fedoraproject.org/f29/httpd
```

- We've named the container `httpd` to make it easier to access later.
- Port 8080 inside the container is redirected to 8080 on the host.
- Notice that we're using an image that binds to a non-privileged (8080) port.

Test the webpage

```
curl localhost:8080
```

Check the process

```
ps -ef | grep podman
```

Check container status and logs

Check the container status and logs

List the running containers

```
podman ps
```

Inspect the (last) ran container - check the Env and IP sections

```
podman inspect -l
```

Check the container logs

```
podman logs httpd
```

Starting and stopping containers

Stop the container and check the status

```
podman stop httpd
```

```
podman ps -a
```

IMAGE	CREATED	STATUS
httpd:latest	5 minutes ago	Exited (0) 2 seconds ago

Start the container back

```
podman start httpd
```

```
podman ps
```

CREATED	STATUS	PORTS	NAMES
7 minutes ago	Up 13 seconds ago	0.0.0.0:8080->8080/tcp	httpd

Using Environment Variables

Search and inspect an image

```
podman search mysql-57-rhel7
skopeo inspect \
  docker://registry.access.redhat.com/rhsc1/mysql-57-rhel7 \
  | grep usage
```

Deploy MySQL

```
podman run --name mysql \
  -e MYSQL_USER=user -e MYSQL_PASSWORD=password \
  -e MYSQL_DATABASE=mydb -e MYSQL_ROOT_PASSWORD=password \
  --detach rhsc1/mysql-57-rhel7:latest
```

Check the logs

```
podman logs mysql
```


Executing a command in a running container with podman-exec

Inspect the environment (different methods show)

```
podman inspect -f '{{ .NetworkSettings.IPAddress }}' mysql
podman exec mysql env
```

Execute a shell inside the mysql container

```
podman exec -it mysql bash
mysql -uroot
    show databases;
    use mydb;

exit
```

Execute a command

```
podman exec -it mysql \
    /opt/rh/rh-mysql57/root/usr/bin/mysql \
    -uuser -ppassword -e 'show databases;'
```

Container Resources

Check processes inside container

```
podman top -l
```

USER	PID	PPID	%CPU	ELAPSED	TTY	TIME	COMMAND
default	1	0	0.000	2m4.13954806s	?	0s	httpd -D FOREGROUND
default	18	1	0.000	2m4.139682033s	?	0s	/usr/bin/coreutils

Display live stream of resource usage statistics

```
podman stats
```

ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET IO	BLOCK IO	PIDS
00b65	httpd	0.07%	40.91MB / 67.4GB	0.06%	-- / --	-- / --	217

Check published ports

```
podman port -l
```

```
8080/tcp -> 0.0.0.0:8080
```

Committing, Saving and Loading Images

Create an image

```
podman run --name ubuntu-apache2 --hostname ubuntu-apache2 \
  --interactive --tty ubuntu:20.04 /bin/bash
```

```
# Install Apache HTTPD
```

```
apt update && apt install -y apache2 && exit
```

List changed files (A - added, C - changed, D - deleted)

```
podman diff ubuntu-apache2
```

Commit image from container with entrypoint and label

```
podman commit --change CMD=/bin/bash --change ENTRYPOINT=/bin/sh \
  --change "LABEL author=c Mihai" ubuntu ubuntu-apache2
```

Save an image

```
podman save -o ubuntu-apache2.tar ubuntu-apache2
```

Load an image

```
podman load -i ubuntu-apache2.tar ubuntu-apache2
```

Modifying the Apache image port from 80 to 8080

Use podman commit to create a custom Apache HTTPD image that listens on port 8080.

Search for the official image and run it

```
podman search httpd --filter=is-official
podman run -it --name httpd-docker docker.io/library/httpd:2.4 /bin/bash
```

Change the port name

```
sed -i 's/Listen 80/Listen 8080/g' /usr/local/apache2/conf/httpd.conf
exit
```

Save to a new image

```
podman stop httpd-docker
podman diff httpd-docker
podman commit -a 'Mihai' httpd-docker cmihai/httpd:2.4
podman images | grep cmihai/httpd
podman rm httpd-docker
```

Test the new image

```
podman run --detach --publish 8080:8080 --name httpd-cmihai docker.io/library/httpd:2.4
curl localhost:8080
```

Tagging or Removing Tags from Images

Tag and image

```
podman tag ubuntu-apache2 cmihai/apache2  
podman tag ubuntu-apache2 cmihai/apache2:latest
```

Remove a tag from an image

```
podman rmi cmihai/apache2:latest
```

Pushing an image to a Registry

Tag the image in your local repository

```
podman tag ubuntu-apache2 quay.io/apache2:latest
```

Push to quay.io

```
podman push quay.io/cmihai/apache2:latest
```

Example build and push

```
podman login quay.io  
podman build --layers=false -t cmihai/jupyterlab:python38 .  
podman tag localhost/cmihai/jupyterlab:python38 \  
    quay.io/cmihai/jupyterlab:latest  
podman push quay.io/cmihai/jupyterlab:latest
```

Create a volume directory on the host and provide permissions

```
mkdir myvol
```

```
podman unshare chown 999:999 myvol
```

<https://www.redhat.com/sysadmin/user-namespaces-selinux-rootless-containers>

Create a container and attach a volume to /data as rw

```
podman run --rm --name ubuntu \  
  --volume ./myvol:/data:Z \  
  --interactive --tty ubuntu /bin/bash
```

SELinux Permissions - Manual approach without using unshare

Let's check what a MySQL container runs as

```
podman run -ti rhsc1/mysql-57-rhel7:latest grep mysql /etc/passwd  
mysql:x:27:27:MySQL Server:/var/lib/mysql:/sbin/nologin
```

Create a directory with owner and group root and give matching permissions

```
sudo mkdir mysql-data && sudo chown -R 27:27 mysql-data
```

Apply the SELinux container_file_t context and policy

```
sudo semanage fcontext -a -t container_file_t './mysql-data(/.*)?'  
sudo restorecon -Rv ./mysql-data
```


Running MySQL with a host directory volume

Start MySQL

```
podman run --name mysql \  
-v ./mysql-data:/var/lib/mysql/data:Z \  
-e MYSQL_USER=user -e MYSQL_PASSWORD=password \  
-e MYSQL_DATABASE=mydb -e MYSQL_ROOT_PASSWORD=password \  
--detach rhscl/mysql-57-rhel7:latest
```

Troubleshoot logs and permissions

```
podman logs mysql; sudo find ./mysql-data; ls -dnZ mysql-data  
  
drwxrwxr-x. 6 100026 100026 system_u:object_r:container_file_t:s0:c303,c890  
4096 Sep 27 09:09 mysql-data
```

Check out the permissions inside the container

```
podman exec mysql ls -lanZ /var/lib/mysql/data  
  
drwxrwxr-x. 27 27 system_u:object_r:container_file_t:s0:c303,c890 .
```

Let's try this one more time with mariadb/server

Stop and remove the container

```
podman stop httpd
podman rm httpd
podman ps -a          # Check that the containers are deleted
```

Removing the container image

```
podman rmi registry.fedoraproject.org/f29/httpd
podman images          # List images
```

Delete everything (stopped containers, pods, dangling images and build cache)

```
podman system prune
```

Setting a container to start at boot using systemd

Enable SELinux and start your container

```
setsebool -P container_manage_cgroup on  
sudo podman run -d --name redis_server -p 6379:6379 redis
```

```
vim /etc/systemd/system/redis-container.service
```

```
[Unit]
```

```
Description=Redis container
```

```
[Service]
```

```
Restart=always
```

```
ExecStart=/usr/bin/podman start -a redis_server
```

```
ExecStop=/usr/bin/podman stop -t 2 redis_server
```

```
[Install]
```

```
WantedBy=local.target
```

Enable the service

```
systemctl enable redis-container.service  
systemctl start redis-container.service  
systemctl stop redis-container.service  
systemctl restart redis-container.service  
systemctl status redis-container.service
```

Creating a Wordpress + MariaDB Server pod

Create a volume and apply permissions

```
mkdir mariadb-data && podman unshare chown 999:999 mariadb-data
```

Create a pod

```
podman pod create --publish 127.0.0.1:8888:80 --name wordpress-pod
```

Start a pod to get info

```
podman run --rm mariadb:latest --verbose --help
```

Start MariaDB with a custom configuration and data volume

```
podman run --name mariadb --pod wordpress-pod \  
-v ./mariadb-data:/var/lib/mysql:Z \  
-e MYSQL_USER=wpsuser -e MYSQL_PASSWORD=password \  
-e MYSQL_DATABASE=wordpress -e MYSQL_ROOT_PASSWORD=password \  
--detach mariadb:10.5.5
```

Start wordpress

```
podman run --name wordpress --pod wordpress-pod \  
-e WORDPRESS_DB_HOST=127.0.0.1:3306 \  
-e WORDPRESS_DB_USER=wpsuser -e WORDPRESS_DB_PASSWORD=password \  
-e WORDPRESS_DB_NAME=wordpress \  
--detach wordpress:latest
```


Building Containers with Podman from a Containerfile

Building container images from Containerfile with podman

Create a Containerfile for Jupyter Lab starting from a UBI

```
FROM registry.access.redhat.com/ubi8/python-38

RUN pip install --upgrade --no-cache-dir jupyterlab

EXPOSE 8888

CMD [ "jupyter", "lab", "--ip=0.0.0.0" ]
```

Build the container image

```
podman build --layers=false -t cmihai/jupyterlab .
```

Test the image

```
podman run --name jupyterlab --detach --publish 8888:8888/tcp cmihai/jupyterlab
podman logs jupyterlab # Retrieve the token to log in
```

Building Containers with Buildah

Building container images with buildah

```
container=$(buildah from fedora)
buildah run ${container} dnf install -y texlive
wget https://github.com/jgm/pandoc/releases/download/2.9.2/pandoc-2.9.2-linux-amd64.tar.gz
tar xzf pandoc-2.9.2-linux-amd64.tar.gz
buildah copy ${container} pandoc-2.9.2/bin /usr/local/bin
buildah commit ${container} cmihai/pandoc
buildah images
buildah inspect ${container}
podman run cmihai/pandoc pandoc --version
```

Pushing images to a container registry with Quay

After creating a quay.io account and password, login using podman

```
podman login quay.io
```

```
Username: cmihai
```

```
Password: ( password here)
```

Creating a new container

Create a new container image based on UBI8 Minimal

```
podman run --name ubi8-httpd \  
  -it registry.access.redhat.com/ubi8/ubi-minimal:latest \  
  /bin/bash  
  
microdnf update -y  
microdnf -y install httpd  
microdnf clean all  
rm -rf /var/cache/yum
```

Commit the container image

Get the container ID (or name) and commit the changes to an image

```
podman ps -l  
podman commit ubi8-httpd quay.io/cmihai/ubi8-httpd:latest
```

Check that the image is there

```
podman images | grep cmihai/ubi8-httpd  
  
quay.io/cmihai/ubi8-httpd latest 8535a6affc3e 15 seconds ago 209 MB
```


Building the same image using a Dockerfile

```
FROM registry.access.redhat.com/ubi8/ubi-minimal
USER root
LABEL maintainer="Mihai Criveti"

# Update image
RUN microdnf update -y && microdnf install -y httpd \
    && microdnf clean all && rm -rf /var/cache/yum \
    && echo "Apache" > /var/www/html/index.html

# Port
EXPOSE 80

# Start the service
CMD ["-D", "FOREGROUND"]
ENTRYPOINT ["/usr/sbin/httpd"]
```

Build and tag the image

```
podman build . -t cmihai/ubi8-httpd:latest
podman tag localhost/cmihai/ubi8-httpd quay.io/cmihai/ubi8-httpd:latest
```

Test the image

Run the image

```
podman run --detach --name httpd --publish 8080:80 quay.io/cmihai/ubi8-httpd:latest
```

Check logs and server status

```
podman logs httpd  
podman port -l  
curl localhost:8080
```

Push the image to quay

Push the local tagged image to quay.io

```
podman push quay.io/cmihai/ubi8-httpd
```

Check the image on quay.io

Check that the image is there

```
podman pull https://quay.io/repository/cmihai/ubi8-httpd
```

Visit the image page:

See: <https://quay.io/repository/cmihai/ubi8-httpd>

Customize the image information

- Create a Description

1. Go to <https://quay.io/repository/cmihai/ubi8-httpd?tab=builds>
2. Upload your build `Dockerfile`